

TemaFinale26 – Sprint 0 (v1)

Introduction

Il sistema **cargoservice** (Protobook cap. 31) automatizza il carico di container nella stiva di una nave tramite un Differential Drive Robot (*cargorobot*). Questo documento, secondo il template del corso, raccoglie per lo **Sprint 0** le sezioni: Requirements, Requirement analysis, Problem analysis, Test plans, Project, Testing, Deployment, Maintenance.

Organizzazione del sistema finale in Sprint

Ogni Sprint costruisce un **sottosistema compiuto** (incrementale):

Sprint	Sottosistema	Modello/codice
Sprint 0 (core)	flusso nominale: loadrequest → riserva slot → IOPort→slot5(marcatura)→slot riservato → HOME (container assunto presente; display/LED come log)	cargoservice26.qak
Sprint 1	aggiunge il sensor dell'IOPort (rilevamento container) e il timeout di 30s	—
Sprint 2	aggiunge Out of service (sospetto guasto del sensor) e il display come web-gui	—
Sprint 3	rifiniture: marker come evento/barcode, gestione delle richieste concorrenti	—

Il **modello qak** del core (Sprint 0) e' gia' nel repository: [cargoservice26/src/cargoservice26.qak](#) (con [Hold](#), [DisplaySim](#), [LedSim](#)). Il POJO Hold e' **compilato ed eseguito** con il test [TestHold](#) (**16 PASS, 0 FAIL**). Vedi le sezioni *Project/Testing*.

Requirements

Testo **esatto del committente** (Protobook, cap. 31 – TemaFinale26):

"A Maritime Cargo shipping company (from now on, simply company) intends to automate the operations of load of containers in the ship's cargo hold (or simply hold). To this end, the company plans to employ a Differential Drive Robot (from now, called cargorobot). The hold is a rectangular, flat area with an Input/Output port (IOPort). The area provides 4 slots to store the containers and a slot named slot5."

- "• The slots1-4 depict the hold areas reserved to store one container each.*
- The slot5 depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a `marker' device labels the container with an identification barcode and signals when this marking activity is completed.*
- The IOPort is a device with a pushbutton and a display. The pushbutton is pressed by the*

customer in order to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold.

- The sensor associated to the IOPort is a device (a sonar) used to detect the presence of a container; when it measures a distance D , such that $D < DFREE/2$, during a reasonable time (e.g. 3 secs)."

TF2026 Requirements – testo esatto:

"The company asks us to build service named cargospace that should work as follows. The cargospace is able to receive a request to load a container sent by some customer by using the pushbutton of the IOPort.

- It sends the answer retrylater, if the IOPort is currently occupied by a container or if the system is Out of service.
- It rejects the request when the hold is already full, i.e. the slots 1-4 are already occupied.
- Otherwise, it considers the system as engaged, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led. When the request to load is accepted, the customer must move the container in the sensor area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes disengaged. Then, the cargospace uses the cargorobot to move the container from the IOPort to the slot5 (for marking the container) and then to the reserved slot. The service must also show on the display on the IOPort:
 - the current state of the hold;
 - the message 'Service working', when it is all is going well;
 - the message 'Out of service' if the sonar sensor measures a distance $D > DFREE$ for at least 3 secs (perhaps a failure of the sonar)."

Requirement analysis

Goal: formalizzare le entita' del dominio togliendo le ambiguita' del linguaggio naturale.

Layout e formalizzazione dell'area

La figura del tema e' **illustrativa** (posizioni relative). Per ragionare senza ambiguita' e rendere il test automatizzabile, l'area si formalizza come **matrice di celle** a coordinate (X, Y) ; lo stato della stiva e' definito da quali celle-slot sono occupate.

	X=0	X=1	X=2	X=3	X=4	X=5	
Y=0	H	.	.	.	I	.	H = HOME (0,0)
Y=1	.	S1	.	.	.	.	I = IOPort (4,0)
...							S1..S5 = slot (occupato se pieno)
...							= cella libera
...							(posizioni autorevoli dalla mappa built-in del DDR, cfr. D1)

Le coordinate **autorevoli** di HOME/IOPort/slot provengono dalla **mappa built-in** del DDR (cfr. **D1**) e dai comandi del docente (Robotsmart26Cmds: IOPort=(4,0), HOME=(0,0)): vanno **lette da quella mappa**, non inventate.

Entita' formalizzate

Container — merce da caricare; riceve un barcode solo durante la marcatura in slot5.

Slot — area di stoccaggio (slot1..slot4 di carico; slot5 di marcatura): **name**, posizione (X,Y), **occupied**, **isMarkingArea**.

Hold — aggregato degli slot: **ioportOccupied()**, **isFull()**, **reserveFreeSlot()**, **confirmStored()**, **freeSlot()** (POJO, vedi [Hold.java](#)).

IOPort — **pushbutton** (input richiesta) + **display** (output). Il **sensor** che rileva il container e' un dispositivo **separato associato** all'IOPort, non parte di esso.

Sensor dell'IOPort (sonar) — misura la distanza D: container presente se $D < D_{FREE}/2$ per $\sim 3s$; sospetto guasto se $D > D_{FREE}$ per $\geq 3s$. Cosa sia esattamente e come fornisca il dato e' da **chiarire col committente**; va tenuto distinto dal sonar del robot in WEnv (estraneo all'applicazione).

Marker — in slot5, etichetta il container e **segnala** il completamento (al cargoservice basta il segnale).

LED — **indicatore di stato** (non un attuatore) che lampeggia mentre il sistema e' *engaged*; collocazione non specificata dai requisiti.

CargoRobot — il DDR fornito. Il committente dispone di *piu' software* (POJO RobotObj26, servizi RobotService26 / robotsmart): quale usare va **analizzato e motivato** (vedi Sprint 1), non solo rinviato.

Macro-parti del sistema

Macro-parte	Fornito / da sviluppare	Natura
cargoservice	da sviluppare	attore (servizio a messaggi)
cargorobot (DDR)	fornito	service esterno (robotsmart)
sensor dell'IOPort	fornito/simulato	sorgente di distanza D
pushbutton+display	forniti/simulati	richiesta / output (web-gui)
marker (slot5)	fornito/simulato	segnala il completamento
LED	fornito/simulato	indicatore di stato (engaged)

Requisiti come user stories

- US1. Come **customer**, voglio inviare una richiesta col pushbutton, per caricare un container.
- US2. Come **customer**, voglio la risposta sul display (reserved | retrylater | reject).
- US3. Come **committente**, voglio il **reject** a stiva piena.

- US4. Come **committente**, voglio **retrylater** se IOPort occupato o Out of service.
- US5. Come **customer**, devo depositare il container entro ~30s, altrimenti disengage.
- US6. Come **committente**, voglio il trasporto IOPort→slot5(marcatura)→slot riservato.
- US7. Come **committente**, voglio il ritorno a HOME a fine ciclo.
- US8. Come **operatore**, voglio vedere stato hold e messaggi sul display.
- US9. Come **committente**, voglio il LED lampeggiante mentre *engaged*.
- US10. Come **committente**, voglio Out of service se il sonar segnala guasto (D>DFREE per ≥3s).

Riferimento: [user stories](#).

Problem analysis

L'analisi del problema (natura dei componenti pojo/service/attore, scelta motivata del DDR, modello delle interazioni, comportamento atteso) e' sviluppata nel documento dedicato: [Sprint 1 \(v1\) – Analisi del Problema](#).

Test plans

Il cargoservice mantiene lo stato della stiva: il test e' **automatizzabile** confrontando lo stato finale con quello atteso (PASS/FAIL).

```
TP1 CARICO NOMINALE (test significativo per la demo)
pre  : available; IOPort libero; almeno uno slot libero (es. slot2)
azione: loadrequest()          -> answer(reserved(slot2))
seq   : moverobot(IOPort) -> moverobot(slot5) -> [marcatura] -> moverobot(slot2) -> moverobot(HOME)
PASS sse: hold.slot("slot2").occupied==true ; robot==HOME ; cargoservice==available

TP2 STIVA PIENA      : slot1..4 occupati -> answer(reject) ; stiva INVARIATA
TP3 RETRYLATER       : IOPort occupato / Out of service -> answer(retrylater) ; stiva INVARIATA
TP4 TIMEOUT (Sprint1) : container non arriva entro 30s -> disengage ; slot liberato
TP5 OUT OF SERVICE (Sprint2): D>DFREE per ≥3s -> display "Out of service"; richiede retrylater
```

TP1 e' il Test Plan automatizzato significativo per la demo (copre l'intero ciclo del core).

Project

Le scelte architetturali (cargoservice come **attore**, cargorobot realizzato da **robotsmart**, natura dei flussi) sono motivate nello [Sprint 1](#). Il **modello qak** del core e' in [cargoservice26.qak](#) (struttura e build modellate su **robotsmart26usage**).

Testing

Stato del testing in questa versione:

- **POJO di dominio (codice eseguito/testato):** Hold e' compilato ed eseguito con il test [TestHold.java](#) — 16 asserzioni sulla logica delle risposte (`reserved/retrylater/reject`, cfr. TP2/TP3), su slot5 (area di marcatura, mai riservata come slot di carico) e sulle posizioni IOPort/HOME. DisplaySim e LedSim compilati.

```
> javac -d build/testclasses utils/domain/Hold.java utils/test/TestHold.java
> java -cp build/testclasses test.TestHold
PASS T3 dopo 4 riserve: stiva piena
PASS T4 slot5 (marcatura) mai riservato    (... 16 asserzioni ...)
TestHold: 16 PASS, 0 FAIL
```

- **Modello qak:** sintassi allineata costruito per costruito ai sorgenti del corso (`robotsmart26tf26.qak`, `firefly`, `robotservice26`). La generazione del codice (plugin QAK nell'IDE), la build e l'esecuzione di **TP1** sono il **passo immediato successivo** (vedi [cargoservice26/README](#)).

Deployment

```
1) docker compose -f ../robotmart26/yamls/unibobasic26.yaml up    (VirtualRobot + GUI + mosquitto)
2) cd ../robotmart26 ; ./gradlew run                             (servizio DDR del docente, 8020)
3) cd ../cargoservice26 ; ./gradlew run                          (cargoservice26, ctxcargo 8030)
4) inviare una loadrequest dal caller del pushbutton
```

Con mosquitto in Docker usare l'**IP reale** del PC (non localhost).

Maintenance

I dispositivi (sensor, display, LED, marker) sono **simulati**; poiche' il cargoservice dipende solo dall'*informazione* scambiata (non dal dispositivo), sostituire un simulato con l'hardware reale (es. il PicoW per il sonar) **non modifica** il cargoservice.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer